Machine Coding: Like Button

- **Problem:** Build a Like button (heart icon + "Like" + count). Clicking it sends a request to the server to like or unlike. While the request is in flight it shows a spinner and ignores further clicks. On success it toggles to "Liked" (filled red) and shows the new count from the server. On failure it keeps the old state and shows the server's error message below the button. Hover turns the border and text red.
- **Tech stack:** React 19 (Vite, TypeScript), plain CSS
- **Time limit:** 60 minutes

PART A: Thought process

The same steps work for any machine coding task:

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases:** every action and its expected result. This is your checklist.
3. **Design the architecture:** components, helpers, services and who owns what.
4. **Design the state:** the smallest set of state; derive the rest.
5. **Design data and contracts:** example data, API shape, component props.
6. **Trace the data flow:** one interaction end to end before writing code.
7. **Find the hard parts:** the 2-4 things that break naive versions.
8. **Plan the build order:** smallest working version first, then one layer at a time.
9. **Cover states and edge cases:** initial, loading, success, error, plus problem-specific ones.
10. **Fix the file structure.**
11. **Write the code.**
12. **Wrap up:** what is handled, trade-offs, what you would add next.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Use cases, architecture, state, contracts	7 min
Build: static button + styles (default, hover, liked)	10 min
Build: mock API + click handler (like / unlike)	10 min
Build: loading spinner, double-click guard, error message	10 min
Build: abort on unmount, accessibility	8 min
Edge cases, walkthrough, questions	10 min

Think out loud. Say what you are deciding and why ("I'll wait for the server before flipping to Liked, because the spec says to show a spinner and the error; an optimistic update would flash Liked and then jump back"). The interviewer grades your reasoning as much as the result, and talking lets them steer you early.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a button with a heart icon, the label "Like" and the like count.
- Default look: grey border and text. Hover: red border and text.
- Click when not liked: send `{ action: "like" }`. Click when liked: send `{ action: "unlike" }`.
- While the request is in flight: replace the heart with a spinner and ignore further clicks.
- On success: toggle to "Liked" (red background, white text) and show the count the server returns.
- On failure: keep the previous state and show the error message from the server below the button.
- The next click clears the old error and tries again.

Non-functional requirements

Area	What it means here
Performance	One request per click. No duplicate requests from double clicks.
Error handling	Network errors and non-2xx responses both end in a readable message; the button never gets stuck spinning.
Accessibility	It is a real `<button>`, toggle state via `aria-pressed`, the error is announced, focus is not lost while loading, visible focus ring.
Reusability	`LikeButton` takes initial `liked` / `count` as props so it works for any post. The API call lives in its own module.
Extensibility	Easy to switch to an optimistic update, add a count animation, or plug in a real endpoint without touching the UI.

Clarifying questions to ask the interviewer

1. Is there a real API, or should I mock it? What is the request and response shape?
2. Should the update be optimistic (flip at once, roll back on error) or wait for the server?
3. Do we show the like count, and does the server return the new count?
4. What should happen to clicks while a request is in flight: ignore, queue, or cancel the old one?
5. Where does the initial state come from: props from the parent (the post data) or a GET on mount?
6. Is there a design for the error (inline text, toast) and should it disappear on its own?

Assumptions we will build with

- No backend: a mock `toggleLike(action)` with a random delay (400-1200 ms) and a 30% failure rate. It has the same contract as a real `POST /api/posts/:id/like`.
- Pessimistic update: the UI changes only after the server says OK (matches "show spinner, then show error").
- Clicks during a request are ignored.
- Initial `liked` and `count` come in as props from the parent.
- The error stays until the next click.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Grey "Like 41" button with heart, no error
Hover / keyboard focus	Red border and text; focus ring for keyboard
Click (not liked)	Spinner replaces heart, request `{ action: "like" }` is sent
Request succeeds	Red filled "Liked 42", spinner gone
Click (liked)	Spinner, request `{ action: "unlike" }`
Request fails	State unchanged, error text shown below, spinner gone
Click after an error	Error cleared, new request sent
Double / triple click fast	Exactly one request
Click while spinner shows	Ignored
Enter / Space on focused button	Same as click (native `<button>`)
Component unmounts mid-request	Request aborted, no state update on an unmounted component

This table is the implementation and test checklist.

---

3. Design the architecture

Module	Responsibility
`App`	Renders the page and passes initial post data to `LikeButton`.
`LikeButton`	Owns liked / count / pending / error state, handles the click, renders the button and the error.
`icons.tsx`	`HeartIcon` and `SpinnerIcon` (inline SVG, `aria-hidden`).
`api/likeApi.ts`	`toggleLike(action, signal)`: the only place that talks to the server (mocked).
`LikeButton.css`	Default, hover, liked, pending, focus and error styles.

```
App
└── LikeButton (state + click handler)
    ├── <button aria-pressed>
    │   ├── HeartIcon | SpinnerIcon
    │   ├── "Like" | "Liked"
    │   └── count
    └── <p role="alert"> error   (only when there is an error)

LikeButton ──calls──> api/likeApi.toggleLike()
```

Start simple: one component plus the API module. A `useLikeMutation` hook only earns its place if two components need the same logic.

4. Design the state / data model

State or field	Purpose
`liked: boolean`	Is the post liked by this user. Decides label, colors, `aria-pressed` and which action to send.
`count: number`	Like count, replaced by the server's value after each success.
`isPending: boolean`	A request is in flight: show spinner, set `aria-busy` / `aria-disabled`.
`error: string | null`	Last error message to show; `null` hides it.
`controllerRef: AbortController | null` (ref)	The in-flight request: guards double clicks synchronously and lets us abort on unmount.

```
const [liked, setLiked] = useState(initialLiked)
const [count, setCount] = useState(initialCount)
const [isPending, setIsPending] = useState(false)
const [error, setError] = useState<string | null>(null)
const controllerRef = useRef<AbortController | null>(null)
```

Senior note

- **Derived, not stored:** the label (`liked ? 'Liked' : 'Like'`), the icon (`isPending ? Spinner : Heart`), the action to send (`liked ? 'unlike' : 'like'`) and all colors (CSS on `aria-pressed`). No `label` or `className` state.
- **Why not one `status` enum?** `'idle' | 'loading' | 'error'` works too. Here `isPending` and `error` are independent enough (we clear the error at the start of a request) that two fields stay simple. Mention both.
- **Ownership:** `LikeButton` owns all four pieces. The parent only gives initial values. If the parent also needs the count (e.g. a header total), lift `liked` / `count` up and pass `onChange` .
- **Ref vs state:** `isPending` is for rendering. The ref is for logic that must be correct before React re-renders (see challenge 1).

---

5. Design the data structures and contracts

Component contract

```
type LikeButtonProps = {
  initialLiked?: boolean // default false
  initialCount?: number  // default 0
}
```

API contract (mocked with the same shape)

```
POST /api/posts/:postId/like
Content-Type: application/json

Request:   { "action": "like" }        // or "unlike"
200 OK:    { "likes": 42 }
500 Error: { "message": "Unknown error during attempted like. Please try again!" }
```

```
export type LikeAction = 'like' | 'unlike'
export type LikeResponse = { likes: number }

toggleLike(action: LikeAction, signal?: AbortSignal): Promise<LikeResponse>
// resolves with the new count, rejects with Error(message) or AbortError
```

Why `action` instead of a plain "toggle" endpoint: if two tabs both send "toggle", they cancel each other out. An explicit `like` / `unlike` is idempotent: sending `like` twice still means liked.

Why the server returns the count: other users like the post too. The server's number is the truth; `count + 1` on the client can drift.

---

6. Trace the data flow

One click on an unliked button, success path:

```
User clicks "Like 41"
        │
        ▼
handleClick()
  controllerRef.current is null → continue (else: ignore click)
  controllerRef.current = new AbortController()
  setIsPending(true), setError(null)
        │
        ▼  re-render: spinner, aria-busy="true", aria-disabled="true"
        │
toggleLike('like', signal)  ──►  mock server (400-1200 ms)
        │
        ▼  resolves { likes: 42 }
setLiked(true), setCount(42)
        │
        ▼  finally
controllerRef.current = null, setIsPending(false)
        │
        ▼  re-render
"Liked 42", red fill, aria-pressed="true", heart back
```

Failure path: the promise rejects → `setError(message)` → `finally` clears pending → button shows the old state and the `<p role="alert">` is read out by screen readers.

---

7. Important technical challenges

Challenge 1: Double click sends two requests

Problem: A naive guard `if (isPending) return` is not enough. Two clicks handled before React re-renders (`click(); click()` in a test, a script, or events queued up on a busy main thread) both run the handler from the same render, where `isPending` is still `false`. Testing this exact version: two back-to-back clicks on "Like 41" sent two `like` requests and showed "Liked 43".

Solution: Guard with a ref, which updates synchronously. The `AbortController` ref doubles as the "request in flight" flag. Keep `isPending` state only for rendering.

```
async function handleClick() {
  if (controllerRef.current) return   // already in flight
  const controller = new AbortController()
  controllerRef.current = controller
  setIsPending(true)
  try { /* ... */ } finally {
    controllerRef.current = null
    if (!controller.signal.aborted) setIsPending(false)
  }
}
```

Challenge 2: Optimistic vs pessimistic update

Problem: Users expect a like to feel instant. But if we flip to "Liked" at once and the request fails (30% here), the button jumps back, and the spec explicitly asks for a spinner and an error.

Solution: Pessimistic by default (wait for the server, show a spinner). Know the optimistic version and when to use it (social feeds where likes rarely fail):

```
// Optimistic variant: flip first, roll back on error
const prev = { liked, count }
setLiked(!liked)
setCount(c => c + (liked ? -1 : 1))
try {
  const { likes } = await toggleLike(liked ? 'unlike' : 'like', signal)
  setCount(likes)            // reconcile with the server
} catch (err) {
  setLiked(prev.liked)       // rollback
  setCount(prev.count)
  setError((err as Error).message)
}
```

Challenge 3: Loading without losing focus or accessibility

Problem: The obvious `disabled={isPending}` makes the browser drop focus from the button. A keyboard user presses Space, the button is disabled, focus jumps to `<body>`, and they lose their place. Screen readers also need to hear "pressed / not pressed" and the error.

Solution: Use `aria-disabled` + the ref guard instead of `disabled`, so the button stays focusable. Use `aria-pressed` for the toggle state (and style off it), `aria-busy` while loading, `aria-hidden` on decorative SVGs, and `role="alert"` on the error so it is announced when it appears.

```
<button
  type="button"
  aria-pressed={liked}
  aria-disabled={isPending}
  aria-busy={isPending}
  onClick={handleClick}
>
```

```
.like-button[aria-pressed='true'] { background: #f00; border-color: #f00; color: #fff; }
.like-button[aria-disabled='true'] { cursor: progress; }
```

Challenge 4: Response after unmount

Problem: The user likes a post in a list and the list re-renders without it (or they navigate away) before the response arrives. The late response calls `setLiked` / `setError` on a component that is gone, and with a real API the request keeps running for nothing.

Solution: Abort the in-flight request in the effect cleanup and skip state updates for an aborted request.

```
useEffect(() => () => controllerRef.current?.abort(), [])

catch (err) {
  if (controller.signal.aborted) return     // unmounted: say nothing
  setError(err instanceof Error ? err.message : 'Something went wrong. Please try again.')
}
```

---

8. Implement in the correct order

#	Step	Working result
1	Clean the Vite template, render `<LikeButton />` in `App`	Empty page with a component
2	Static button: heart SVG + "Like" + count	Button visible
3	CSS: default grey, hover red, `[aria-pressed='true']` red fill, focus ring	All looks done, no logic
4	`liked` state + local toggle on click (no API yet)	Like / Liked toggles

#	Step	Working result
5	`api/likeApi.ts` mock: delay, random failure, returns `{ likes }`	Contract ready
6	Async `handleClick`: call API, update `liked` and `count` on success	Talks to the "server"
7	`isPending` + spinner icon	Loading state visible
8	`error` state + `<p role="alert">`, clear on next click	Failure path done
9	Ref guard for double clicks, `aria-disabled` instead of `disabled`	One request per click, focus kept
10	`AbortController` + cleanup on unmount	No late updates
11	Walk through the use-case table (success, failure, retry, double click, keyboard)	Verified

9. Cover all states and edge cases

State / edge case	What the UI does
Initial (not liked)	Grey "Like 41", heart icon
Initial (liked, from props)	Red filled "Liked 41"
Hover / focus	Red border and text; blue focus ring for keyboard
Loading	Spinner, `aria-busy="true"`, clicks ignored, focus stays
Success (like)	"Liked", count from the server
Success (unlike)	"Like", count from the server
Error	Old state kept, error text below, spinner gone
Retry after error	Error cleared at click time, new request
Double / triple click	One request (ref guard)
Click while loading	Ignored
Unmount while loading	Request aborted, no state update
Server count differs from local (others liked)	Server value wins
Non-JSON or empty error body (real API)	Fallback message "Something went wrong. Please try again."
Reduced motion preference	Spinner rotates slower

10. Final structure

```
_31_like_button/
├── index.html
├── package.json
└── src/
    ├── main.tsx
    ├── App.tsx                    # page, passes initial post data
    ├── index.css                  # page layout
    ├── api/
    │   └── likeApi.ts             # toggleLike(action, signal): mock + real fetch version
    └── components/
        └── LikeButton/
            ├── LikeButton.tsx     # state, click handler, markup
            ├── LikeButton.css     # default / hover / liked / loading / error styles
            └── icons.tsx          # HeartIcon, SpinnerIcon
```

Delete the template's `App.css` and `assets/react.svg`; they are no longer used.

---

11. Full working code

`src/api/likeApi.ts`

```
export type LikeAction = 'like' | 'unlike'
export type LikeResponse = { likes: number }

// shortcut: in-memory "server" with random failures, swap for the real fetch below when there is a
backend
let serverLikes = 41
const FAILURE_RATE = 0.3

export function toggleLike(action: LikeAction, signal?: AbortSignal): Promise<LikeResponse> {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      if (Math.random() < FAILURE_RATE) {
        reject(new Error('Unknown error during attempted like. Please try again!'))
        return
      }
      serverLikes += action === 'like' ? 1 : -1
      resolve({ likes: serverLikes })
    }, 400 + Math.random() * 800)

    signal?.addEventListener('abort', () => {
      clearTimeout(timer)
      reject(new DOMException('Request aborted', 'AbortError'))
    })
  })
}

// Real API version (same contract):
// export async function toggleLike(action: LikeAction, signal?: AbortSignal): Promise<LikeResponse> {
//   const res = await fetch('/api/posts/1/like', {
//     method: 'POST',
//     headers: { 'Content-Type': 'application/json' },
//     body: JSON.stringify({ action }),
//     signal,
//   })
//   const data = await res.json()
//   if (!res.ok) throw new Error(data.message ?? 'Request failed')
//   return data
// }
```

`src/components/LikeButton/icons.tsx`

```
export function HeartIcon() {
  return (
    <svg className="like-icon" viewBox="0 0 24 24" fill="currentColor" aria-hidden="true">
      <path d="M12 21.35l-1.45-1.32C5.4 15.36 2 12.28 2 8.5 2 5.42 4.42 3 7.5 3c1.74 0 3.41.81 4.5
2.09C13.09 3.81 14.76 3 16.5 3 19.58 3 22 5.42 22 8.5c0 3.78-3.4 6.86-8.55 11.54L12 21.35z" />
    </svg>
  )
}

export function SpinnerIcon() {
  return (
    <svg className="like-icon spinner" viewBox="0 0 24 24" fill="none" aria-hidden="true">
      <circle cx="12" cy="12" r="9" stroke="currentColor" strokeWidth="3" opacity="0.25" />
      <path d="M21 12a9 9 0 0 0 0-9-9" stroke="currentColor" strokeWidth="3" strokeLinecap="round" />
    </svg>
  )
}
```

`src/components/LikeButton/LikeButton.tsx`

```
import { useEffect, useRef, useState } from 'react'
import { toggleLike } from '../../api/likeApi'
import { HeartIcon, SpinnerIcon } from './icons'
import './LikeButton.css'

type LikeButtonProps = {
  initialLiked?: boolean
  initialCount?: number
}

export default function LikeButton({ initialLiked = false, initialCount = 0 }: LikeButtonProps) {
  const [liked, setLiked] = useState(initialLiked)
  const [count, setCount] = useState(initialCount)
  const [isPending, setIsPending] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const controllerRef = useRef<AbortController | null>(null)

  // Cancel an in-flight request if the button unmounts
  useEffect(() => () => controllerRef.current?.abort(), [])

  async function handleClick() {
    // A ref, not isPending: two clicks in the same tick both still see isPending === false
    if (controllerRef.current) return

    const controller = new AbortController()
    controllerRef.current = controller
    setIsPending(true)
    setError(null)

    try {
      const { likes } = await toggleLike(liked ? 'unlike' : 'like', controller.signal)
      setLiked(!liked)
      setCount(likes)
    } catch (err) {
      if (controller.signal.aborted) return
      setError(err instanceof Error ? err.message : 'Something went wrong. Please try again.')
    } finally {
      controllerRef.current = null
      if (!controller.signal.aborted) setIsPending(false)
    }
  }

  return (
    <div className="like-wrapper">
      <button
```

```
        type="button"
        className="like-button"
        aria-pressed={liked}
        aria-disabled={isPending}
        aria-busy={isPending}
        onClick={handleClick}
      >
        {isPending ? <SpinnerIcon /> : <HeartIcon />}
        <span>{liked ? 'Liked' : 'Like'}</span>
        <span className="like-count" aria-label={`${count} likes`}>
          {count}
        </span>
      </button>

      {error && (
        <p className="like-error" role="alert">
          {error}
        </p>
      )}
    </div>
  )
}
```

`src/components/LikeButton/LikeButton.css`

```
.like-wrapper {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
}

.like-button {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 6px 14px;
  border: 2px solid #888;
  border-radius: 999px;
  background: #fff;
  color: #888;
  font: 600 16px/1 system-ui, sans-serif;
  cursor: pointer;
  transition: color 0.15s, border-color 0.15s, background-color 0.15s;
}

.like-button:hover {
  border-color: #f00;
  color: #f00;
}

.like-button[aria-pressed='true'] {
  border-color: #f00;
  background: #f00;
  color: #fff;
}

.like-button[aria-disabled='true'] {
  cursor: progress;
}

.like-button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 2px;
}

.like-count {
  padding-left: 6px;
  border-left: 1px solid currentColor;
  font-variant-numeric: tabular-nums;
```

```
}

.like-icon {
  width: 20px;
  height: 20px;
}

.spinner {
  animation: spin 0.8s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: reduce) {
  .spinner {
    animation-duration: 2.4s;
  }
}

.like-error {
  margin: 0;
  color: #d00;
  font-size: 14px;
}
```

src/App.tsx

```
import LikeButton from './components/LikeButton/LikeButton'

export default function App() {
  return (
    <main className="app">
      <h1>Like Button</h1>
      <LikeButton initialLiked={false} initialCount={41} />
    </main>
  )
}
```

`src/index.css`

```
body {
  margin: 0;
  min-height: 100vh;
  display: grid;
  place-items: center;
  font-family: system-ui, sans-serif;
  background: #f5f5f5;
  color: #222;
}

.app {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px;
}
```

`src/main.tsx` stays as generated by Vite (it imports `index.css` and renders `<App />` in `StrictMode`).

---

12. Wrap-up

Edge cases handled

- Success, failure and retry, with the server's count as the source of truth.
- Fast double clicks send exactly one request (ref guard, not state).
- The button never gets stuck: `finally` always clears the pending state.
- Focus is kept while loading (`aria-disabled` instead of `disabled`).
- Late responses after unmount are aborted and ignored.
- Screen readers hear the toggle state (`aria-pressed`) and the error (`role="alert"`).

Trade-offs to mention to the interviewer

- **Pessimistic vs optimistic:** pessimistic is honest and matches the spec, but feels slower. Optimistic feels instant but needs rollback and can flicker on errors.
- **Ignore clicks while pending vs cancel and resend:** ignoring is simplest and avoids races. Cancel-and-resend lets users change their mind, but an aborted fetch may still have reached the server, so you need idempotent `like` / `unlike`.
- **Local state vs global cache:** local state is enough for one button. If the same post shows in several places (feed and detail page), use a shared cache (React Query / Redux) so they stay in sync.
- **Mock with `Math.random`:** good for a demo, but not deterministic. In tests, stub `toggleLike` instead.

What to add with more time

- Tests with React Testing Library: success, failure, retry, double click sends one request, unmount aborts.
- Optimistic mode as a prop, with rollback.

- Count animation (number flip) and a heart "pop" on like.
- Debounce rapid like/unlike toggles into one final request for very spammy users.
- Format big counts (`1.2K`) with `Intl.NumberFormat` in compact notation.
- Real API with retries and backoff for network errors (not for 4xx).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just toggle `liked` on the client and fire the request in the background?

That is an optimistic update without rollback. If the request fails, the UI lies: it shows "Liked" while the server says not liked. The spec asks for a spinner and an error message, which only makes sense if we wait for the server. If I do go optimistic, I keep the previous value and roll back on error.

Q2. Why send `"like"` / `"unlike"` instead of a single `"toggle"` action?

`toggle` is not idempotent. If a request is retried or sent from two tabs, two toggles cancel out. With explicit actions, sending `like` twice still ends in "liked", so retries are safe.

Q3. Where should the initial liked state come from?

From the post data the parent already loaded (`post.likedByMe` , `post.likeCount`), passed as props. A separate GET per button would be N extra requests in a feed. If the button must be standalone, fetch on mount and show a skeleton until it arrives.

State design

Q4. Why `isPending` in state *and* the controller in a ref? Isn't that duplicate state?

They serve different jobs. State drives rendering (spinner, `aria-busy`), and React updates it on the next render. The ref is read synchronously inside the handler, so a second click in the same tick sees the in-flight request. Using only state lets duplicate requests through; using only a ref would not re-render the spinner.

Q5. Why not compute the count as `count + 1` after a successful like?

Other users like the post at the same time. The server returns the real total, so I trust that. `count + 1` is fine for an optimistic preview, but I still replace it with the server value when the response comes in.

Q6. Would you use `useReducer` here?

With four fields and one async flow, `useState` is clear enough. If I add optimistic mode, rollback and retry, a reducer with actions like `REQUEST` , `SUCCESS` , `FAILURE` makes the transitions explicit and easy to unit test without rendering.

Double click / concurrency

Q7. React 18 batches and flushes updates for click events. Why is `if (isPending) return` still not enough?

React flushes a click's update in a microtask after the handler, so two separate user clicks are usually safe. But any clicks handled before that flush (two `click()` calls in a test or script, or events queued behind a long task) run the handler from the same render, and both closures see `isPending === false`. A ref is written synchronously, so it is correct no matter when the re-render happens.

Q8. What if the user wants to unlike while the like request is still in flight?

Option A (ours): ignore clicks while pending. Option B: abort the current request and send the new action; with idempotent `like` / `unlike` the last request wins on the server. Option C: queue the desired final state and send only that when the current request ends. B or C is better for very fast users; A is simplest and correct.

Q9. The user spams like/unlike 10 times. How do you avoid 10 requests?

Update the UI optimistically on each click, and debounce the network call (for example 500 ms). After the last click, send only the final state if it differs from the server's state. That is one request instead of ten.

Accessibility

Q10. Why not use `disabled` while loading?

A disabled button cannot keep focus, so the browser moves focus to `<body>` and a keyboard user loses their place. `aria-disabled="true"` keeps it focusable and tells screen readers it is unavailable; the ref guard blocks the click.

Q11. How does a screen reader user know the button is liked?

`aria-pressed` makes it a toggle button, so it is read as "Like, toggle button, pressed / not pressed". The visible label also changes to "Liked". The icons are `aria-hidden` so they are not read as "image".

Q12. How is the error announced?

The error `<p>` has `role="alert"`, an assertive live region. It is inserted only when there is an error, so screen readers announce the message as soon as it appears, without moving focus.

Errors and network

Q13. What if the API is slow (5+ seconds)?

The spinner covers it, and `aria-busy` tells assistive tech. I would add a timeout with `AbortSignal.timeout(8000)` and show "Taking too long, try again". For a feed, an optimistic update hides the slowness completely.

Q14. What if the API returns HTML or an empty body on error?

`res.json()` would throw. Wrap it: `const data = await res.json().catch(() => ({}))`, then `throw new Error(data.message ?? 'Request failed')`. The component already falls back to a generic message for anything that is not an `Error`.

Q15. Should you retry automatically on failure?

Only for network errors and 5xx, with a small number of retries and exponential backoff, and only because `like` / `unlike` are idempotent. Never retry 4xx (bad request, not logged in). Here the user retries by clicking again, which is enough for a like.

Performance and scale

Q16. What if there are 1,000 like buttons in a feed?

Each button is tiny and owns its state, so a click re-renders only that button. Keep the list virtualized, pass stable props, and wrap `LikeButton` in `React.memo` if the parent re-renders often. Avoid one global "likes" object in Context that re-renders every button on each like.

Q17. The same post is shown in the feed and in an open modal. How do they stay in sync?

Local state cannot do it. Move the like data into a shared cache keyed by post id (React Query with `setQueryData`, or a store). Both buttons read from it, and the mutation updates the cache once.

Trade-offs and "why not X"

Q18. Why not put the fetch inside a `useEffect` that watches `liked`?

A like is an event, not a sync with external state. Effects would also fire on mount and in StrictMode twice, sending unwanted requests. Event handlers are the right place for mutations; effects are for subscriptions and cleanup (like our abort).

Q19. Why inline SVG icons instead of an icon library or an emoji?

Two icons do not justify a dependency. Inline SVG uses `currentColor`, so the heart follows the text color in every state (grey, red, white) with no extra CSS. Emoji look different on each OS and cannot be recolored.

Q20. How would you test this?

With React Testing Library and a mocked `toggleLike`: click → spinner shows → resolve → "Liked" and new count; reject → error shown and state unchanged; click twice → mock called once; unmount during a pending request → no warnings and the signal is aborted. Query by role: `getByRole('button', { pressed: true })`.